

Experiment No. 5

Aim:

To use Burp Proxy to test Web Applications.

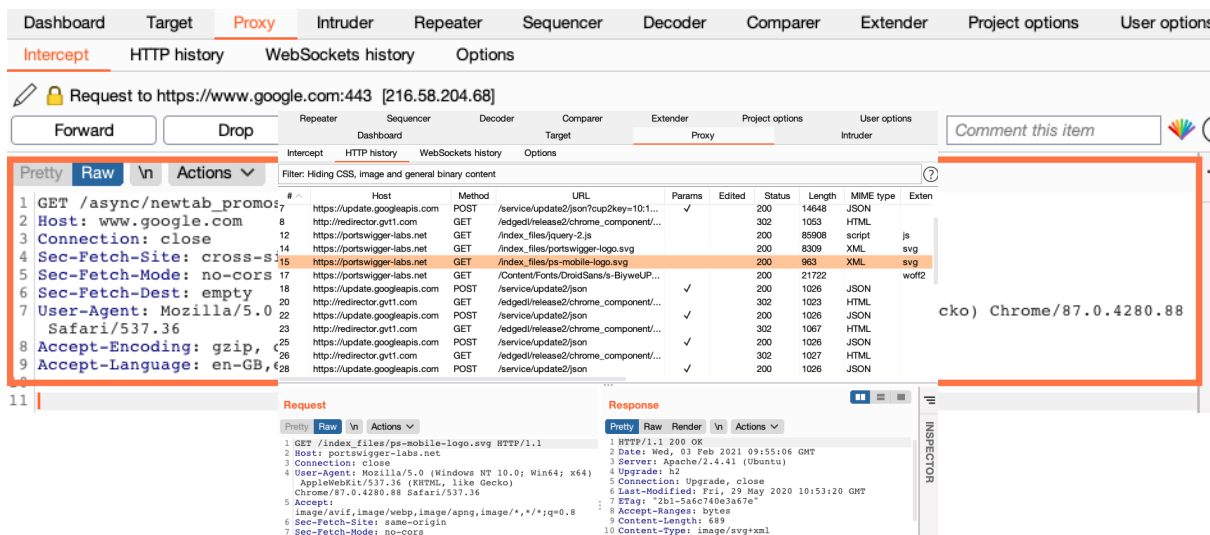
Theory:

To use Burp for [penetration testing](#), use [Burp's browser](#), which requires no additional configuration. To launch Burp's browser, go to the **Proxy > Intercept** tab and click **Open Browser**. A new browser session will open in which all traffic is proxied through Burp automatically. You can even use this to test using HTTPS.

Once you have Burp running and have opened [Burp's browser](#), go to the **Proxy > Intercept** tab, and ensure that interception is turned on (if the button says **Intercept is off** then click it to toggle the interception status). Then, go to the browser and visit any URL.

Each HTTP request made by the browser is displayed in the **Intercept** tab. You can view each message, and edit it if required. When you are done making changes, click the **Forward** button to send the request on to the destination web server. If at any time there are intercepted messages pending, you will need to forward all of these in order for the browser to complete loading the pages it is waiting for.

You can toggle the **Intercept is on / off** button in order to browse normally without any interception, if you require. For more help, see [Getting started with Burp Proxy](#).



As you browse an application with Burp running, the **Proxy > HTTP history** tab keeps a record of all requests and responses, even while the intercept feature is turned off. From this tab, you can review the series of requests you have made. Select an item in the table to view the full request and response in the [message editor](#) panel.

For editable messages, such as in Burp Repeater, you can also make changes to this decoded value in the Inspector. The relevant encodings will automatically be reapplied to the value as you type.

Repeater
Sequencer
Decoder
Target
Extender
Proxy
Project options
User options

Dashboard
WebSockets history
Options
Intruder

Intercept
HTTP history
WebSockets history
Options

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status	Length	MIME type	Extension
6	https://www.gstatic.com	GET	/generate_204			204	102		
7	https://update.googleapis.com	POST	/service/update/3on?cup2key=101...	✓		200	14648	JSON	
8	https://redirector.gvt1.com	GET	/edged/relase2/chrome_component/...			302	1053	HTML	
12	https://portswigger-labs.net	GET	/index_files/query-2.js			200	85908	script	js
14	https://portswigger-labs.net	GET	/index_files/portswigger-logo.svg			200	8309	XML	svg
15	https://portswigger-labs.net	GET	/index_files/js-mobile-logo.svg			200	963	XML	svg
17	https://portswigger-labs.net	GET	/ContentFonts/DroidSans/s-BiyewUp...			200	21722		woff2
18	https://update.googleapis.com	POST	/service/update/3on	✓		200	1026	JSON	
20	https://redirector.gvt1.com	GET	/edged/relase2/chrome_component/...			302	1023	HTML	
22	https://redirector.gvt1.com	POST	/service/update/3on	✓		200	1026	JSON	
23	https://redirector.gvt1.com	GET	/edged/relase2/chrome_component/...			302	1067	HTML	
25	https://update.googleapis.com	POST	/service/update/3on	✓		200	1026	JSON	
26	https://redirector.gvt1.com	GET	/edged/relase2/chrome_component/...			302	1027	HTML	

Request
Response
Inspector

Pretty
Raw
In
Actions

1 POST /service/update/2/3on?cup2k
2 Host: https://update.googleapis.com
3 Connection: close
4 Content-Length: 1929
5 X-Goog-Update-AppId: emahnhpold
6 X-Goog-Update-AppId: emahnhpold
7 X-Goog-Update-AppId: emahnhpold
8 Content-Type: application/json
9 Rec-Fetch-Method: no-cors
10 Rec-Fetch-Test: empty
11 Rec-Fetch-Test: empty
12 Rec-Fetch-Test: empty

Pretty
Raw
Render
In
Actions

1 HTTP/1.1 200 OK
2 Content-Security-Policy: script
3 Cache-Control: no-cache, no-sto
4 Pragma: no-cache
5 Expires: Mon, 01 Jan 1990 00:00
6 Date: Wed, 01 Feb 2021 05:19:05
7 X-Cup-Server-Proof: 30460221008
8 ETag: W/"30460221008259c5b527c
9 Content-Type: application/json
10 X-Daynam: 5147
11 X-Daystart: 6905
12 X-Content-Type-Options: nosniff

Inspector

Query Parameters (2)
NAME VALUE
cup2key 101548213833
cup2req 91f875655221e5a1120...
Request Headers (12)
Response Headers (17)

As you browse, Burp also builds up a site map of the target application by default. You can view this on the **Target > Site map** tab.

The site map contains all of the URLs you have visited in the browser, and also all of the content that Burp has inferred from responses to your requests (e.g. by parsing links from HTML responses). Items that have been requested are shown in black, and other items are shown in gray. You can expand branches in the tree, select individual items, and view the full requests and responses (where available).

For more help, see [Using the Target tool](#). You can control which content gets added to the site map as you browse by configuring a suitable [live scanning task](#).

Repeater

Sequencer

Decoder

Composer

Extender

Project options

User options

Dashboard

Target

Proxy

Intruder

Site map

Scope

Issue definitions

Filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

Contents

Host	Method	
https://portswigger-labs...	GET	/
https://portswigger-labs...	GET	/cors.ph
https://portswigger-labs...	GET	/cors.ph
https://portswigger-labs...	GET	/cors.ph
https://portswigger-labs...	GET	/crossdc
https://portswigger-labs...	GET	/csp/
https://portswigger-labs...	GET	/csp/?C:
https://portswigger-labs...	GET	/csp/?C:
https://portswigger-labs...	GET	/csp/?C:
https://portswigger-labs...	GET	/csp/?C:
https://portswigger-labs...	GET	/csp/?C:

Issues

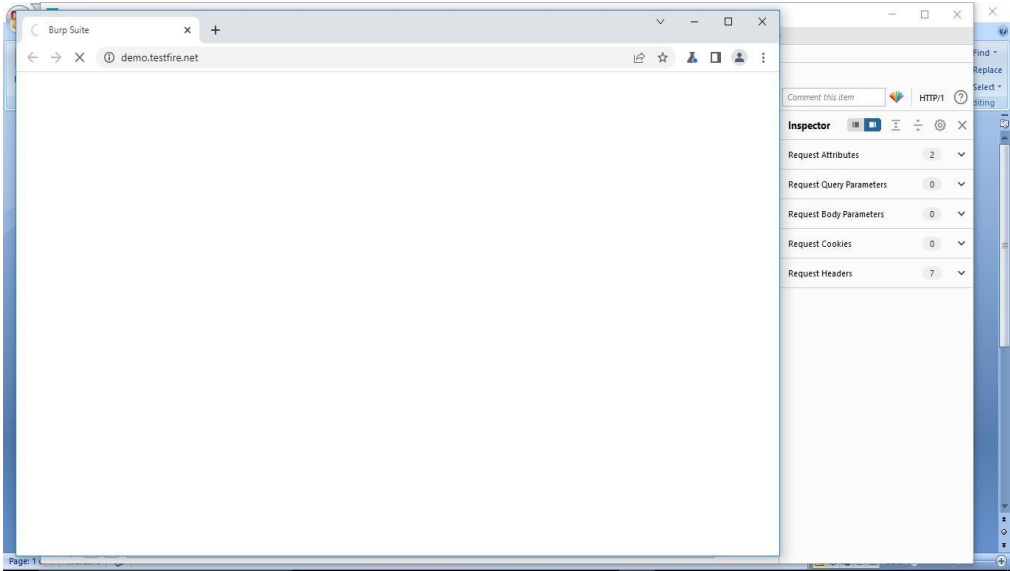
- Cross-site scripting (reflected) [2]
- Flash cross-domain policy
- External service interaction (DNS)
- Cross-site scripting (DOM-based) [2]
- Serialized object in HTTP message [2]
- Strict transport security not enforced [4]
- Link manipulation (DOM-based)
- Open redirection (DOM-based)
- Cross-origin resource sharing [2]
- Cross-origin resource sharing: arbitrary c
- Input returned in response (reflected) [2]
- Cross-domain Referer leakage

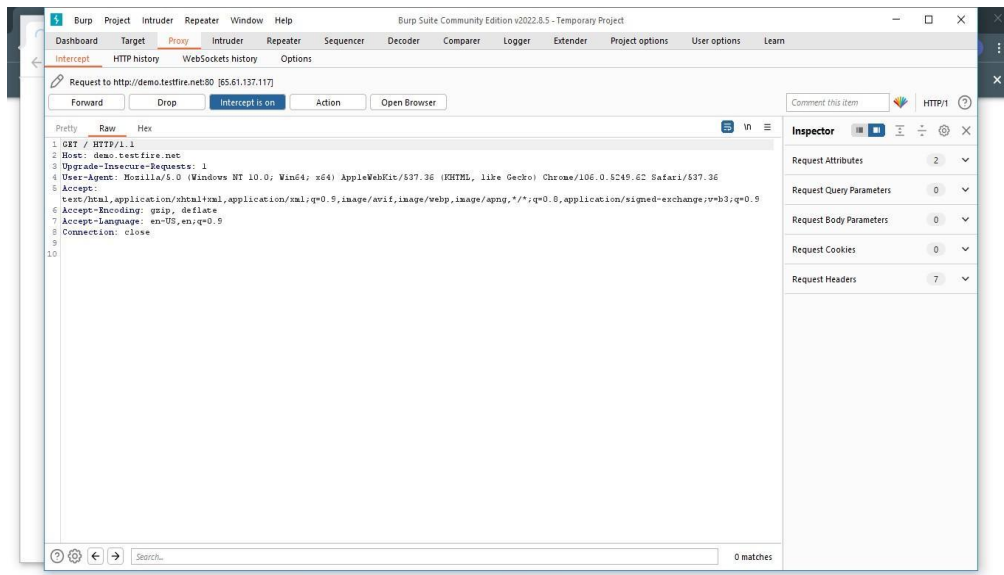
Burp Suite is designed to be a hands-on tool, where the user controls the actions that are performed. At the core of Burp's penetration testing workflow is the ability to pass HTTP requests between the Burp tools in order to carry out particular tasks.

You can send messages from the **Proxy > Intercept**, **HTTP history**, or **Site map** tabs, and indeed anywhere else in Burp that you see HTTP messages. To do this, select one or more messages, and use the context menu to send the request to another tool.

Repeater		Sequencer		Decoder		Comparer		Extender		Project options		User options			
Dashboard				Target				Proxy				Intruder			
Intercept		HTTP history		WebSockets history		Options									
Filter: Hiding CSS, image and general binary content													?		
# ^	Host	Method	URL		Params	Edited	Status	Length	MIME type	Exten					
6	http://www.gstatic.com	GET	/generate_204				204	102							
7	https://update.googleapis.com	POST	/service/update2/json?cup2key=10-1		✓		200	14648	JSON						
8	http://redirector.gvt1.com		https://update.googleapis.co...23d0cd2f293cdf8711b2d13613ab				302	1053	HTML						
12	https://portswigger-labs.net		Add to scope				200	85908	script			js			
14	https://portswigger-labs.net		Scan				200	8309	XML			svg			
15	https://portswigger-labs.net						200	963	XML			svg			
17	https://portswigger-labs.net		Do passive scan				200	21722				woff2			
18	https://update.googleapis.c		Do active scan				200	1026	JSON						
20	http://redirector.gvt1.com		Send to Intruder		^%		302	1023	HTML						
22	https://update.googleapis.c		Send to Repeater		^%R		200	1026	JSON						
23	http://redirector.gvt1.com						302	1067	HTML						
25	https://update.googleapis.c		Send to Sequencer				200	1026	JSON						
26	http://redirector.gvt1.com		Send to Comparer (request)				302	1027	HTML						
			Send to Comparer (response)												

Output:





Conclusion:

Thus, I have learnt to use Burp proxy to test web application.